

Lec20

Advanced DOM Manipulation

Basic DOM manipulation means:

- Selecting elements
- Changing text
- Changing styles

Advanced DOM manipulation means:

- Traversing DOM
- Creating dynamic elements
- Removing elements
- Replacing elements
- Event delegation

DOM Traversing

Traversing means moving between elements.

Example:

```
<div id="parent">  
  <h1 id="heading">Hello</h1>  
  <p>Paragraph</p>  
</div>
```

parentElement

Move to parent.

```
let heading = document.getElementById("heading");  
console.log(heading.parentElement);
```

Output:

```
<div id="parent">...</div>
```

children

Access child elements.

```
let parent = document.getElementById("parent");  
console.log(parent.children);
```

firstElementChild

```
console.log(parent.firstElementChild);
```

lastElementChild

```
console.log(parent.lastElementChild);
```

nextElementSibling

```
console.log(heading.nextElementSibling);
```

previousElementSibling

```
console.log(heading.previousElementSibling);
```

Creating Elements Dynamically

Create element using JavaScript.

```
let newDiv = document.createElement("div");
```

Add content:

```
newDiv.innerText = "Hello Students";
```

Append to body:

```
document.body.appendChild(newDiv);
```

append vs prepend

append → adds at end

```
parent.append(newDiv);
```

prepend → adds at beginning

```
parent.prepend(newDiv);
```

Remove Elements

```
element.remove();
```

Example:

```
document.getElementById("heading").remove();
```


Replace Elements

```
oldElement.replaceWith(newElement);
```

Example:

```
let newHeading = document.createElement("h2");
```

```
newHeading.innerText = "New Heading";
```

```
document.getElementById("heading").replaceWith(newHeading);
```

Event delegation

Suppose you have many buttons.

Bad approach:

```
btn1.addEventListener(...)
```

```
btn2.addEventListener(...)
```

```
btn3.addEventListener(...)
```

Better approach:

Attach event to parent.

Example

```
<ul id="list">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
document.getElementById("list").addEventListener("click", function(event) {
  console.log(event.target);
});
```

Why useful?

- Better performance
- Handles dynamic elements

Form Validation

Form validation means:

Checking user input before submission.

Example:

- Name required
- Email valid
- Password length
- Phone number format

Example

```
<form id="form">
```

```
<input id="name" type="text">
```

```
<input id="email" type="text">
```

```
<button>Submit</button>
```

```
</form>
```

Why Validation?

Without validation:

- Empty forms submitted
- Invalid emails
- Weak passwords

Validation improves:

- Security
- User experience
- Data quality

Validation Types

HTML Validation

JavaScript Validation

HTML Validation

```
<input type="email" required>
```

Browser automatically checks.

Javascript Validation

More powerful.

Example:

```
let name = document.getElementById("name").value;
```

```
if(name === "") {  
    alert("Name required");  
}
```

Common Validation Rules

Required Field

```
if(name.trim() === "") {  
    alert("Field cannot be empty");  
}
```

Email Validation

Simple check:

```
if(!email.includes("@")) {  
  alert("Invalid email");  
}
```

Using Regex:

```
let pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
if(!pattern.test(email)) {  
  alert("Invalid email");  
}
```

Password Validation

Example:

- Minimum 8 characters
- One uppercase
- One number

```
if(password.length < 8) {  
    alert("Weak password");  
}
```

Prevent Form Submission

Default form submits and refreshes page.

Prevent it:

```
form.addEventListener("submit", function(event){  
    event.preventDefault();  
});
```

Very important.

Example Validation

```
document.getElementById("form").addEventListener("submit", function(event){  
    event.preventDefault();
```

```
    let name = document.getElementById("name").value;
```

```
    if(name.trim() === "") {  
        alert("Name is required");  
    } else {  
        alert("Form Submitted");  
    }  
});
```

localStorage

localStorage allows browser to store data permanently.

Data remains even after:

- Refresh
- Browser restart

Storage format:

- Key → Value

Important:

- Stores only strings

Why localStorage?

Useful for:

- Todo list
- Theme settings
- Notes app
- Cart data

localStorage Methods

setItem()

Store data.

```
localStorage.setItem("username", "Naman");
```

getItem()

Retrieve data.

```
localStorage.getItem("username");
```

removeItem()

Remove one item.

```
localStorage.removeItem("username");
```

clear()

Remove everything.

```
localStorage.clear();
```

Example

Save name:

```
localStorage.setItem("name", "Naman");
```

Read:

```
console.log(localStorage.getItem("name"));
```

Storing Objects in localStorage

Problem:

localStorage stores only strings.

Wrong:

```
localStorage.setItem("user", {name:"Naman"});
```

Solution:

Use JSON.

Convert object → string

```
let user = {  
  name: "Naman",  
  age: 22  
};
```

```
localStorage.setItem("user", JSON.stringify(user));
```

Convert string → object

```
let data = JSON.parse(localStorage.getItem("user"));  
console.log(data.name);
```

sessionStorage

Similar to localStorage.

Difference:

- localStorage → permanent
- sessionStorage → removed when tab closes

Example:

```
sessionStorage.setItem("temp", "hello");
```

Dynamic UI

Dynamic UI means:

Interface changes automatically based on data or user actions.

Examples:

- Dark mode toggle
- Add/remove todo
- Live search
- Shopping cart updates

Without page reload.

Example: Dynamic Todo List

HTML:

```
<input id="task">
```

```
<button id="addBtn">Add</button>
```

```
<ul id="list"></ul>
```

```
let input = document.getElementById("task");  
let btn = document.getElementById("addBtn");  
let list = document.getElementById("list");
```

```
btn.addEventListener("click", function(){  
  let task = input.value;
```

```
  if(task.trim() === "") return;
```

```
  let li = document.createElement("li");  
  li.innerText = task;
```

```
  list.appendChild(li);  
  input.value = "";  
});
```


Dynamic Styling

Example:

```
button.addEventListener("click", function(){  
    document.body.classList.toggle("dark");  
});
```

CSS:

```
.dark {  
    background: black;  
    color: white;  
}
```

Task

Add, update, delete, and store data in browser and make to do list using DOM manipulation and take values from form and validate.